

Multiplicador 4×4 sem sinal, com detecção de overflow

Descrição estrutural em Alliance CAD (genlib), verificação exaustiva com asimut, layout físico (ocp/nero/graal) e atraso de propagação medido no nível lógico e por conferência SPICE (spiceopus).

1. Objetivo

Projetar um multiplicador combinacional de dois operandos de 4 bits sem sinal (A, B entre 0 e 15), produzindo o produto de 8 bits $P = A \times B$ e um sinal OVF que indica overflow ($P > 15$). O circuito foi descrito estruturalmente, porta por porta, usando a linguagem genlib do Alliance CAD, e verificado contra as 256 combinações possíveis de entrada.

2. Arquitetura

A implementação segue o modelo de array multiplier: uma matriz de produtos parciais gerada por 16 portas AND2 (a2_x2), somada por diagonais com meios-somadores e somadores completos (xr2_x1, o2_x2) até produzir os oito bits do produto. A primeira diagonal é somada por meios-somadores; as diagonais seguintes, por somadores completos, propagando o carry até a última linha, que fecha P4 a P7. Os oito bits de saída passam por buffers (buf_x2) até os conectores externos. OVF é o OR dos quatro bits mais altos do produto (P4 a P7).

3. Ambiente e ferramentas

O fluxo roda em Docker (Ubuntu 18.04 + Alliance CAD 5.1.1), com scripts para cada etapa (build_mult4.sh, run.sh, place_route.sh, find_delay.sh). Alguns problemas do pacote precisaram ser contornados: o binário asimut amd64 trava com segmentation fault em qualquer netlist estrutural com instanciação de componente (corrigido usando a variante i386 do pacote); o gerador de VHDL do genlib produz identificadores de barramento inválidos (corrigido após a geração); as ferramentas de layout físico (ocp, nero, graal) esperam o arquivo de tecnologia em um caminho diferente de onde o pacote instala; e o SpiceOpus exige uma versão do Ubuntu mais nova que a do restante do fluxo, rodando por isso em um container separado.

4. Metodologia de verificação

O resultado de cada um dos 256 casos simulados no asimut foi comparado com o valor esperado em mult4_table.txt, uma tabela de referência simples (A, B, P, OVF) com uma linha por combinação.

5. Resultados

Todos os 256 vetores de teste passaram — nenhuma divergência em P ou OVF contra a tabela de referência.

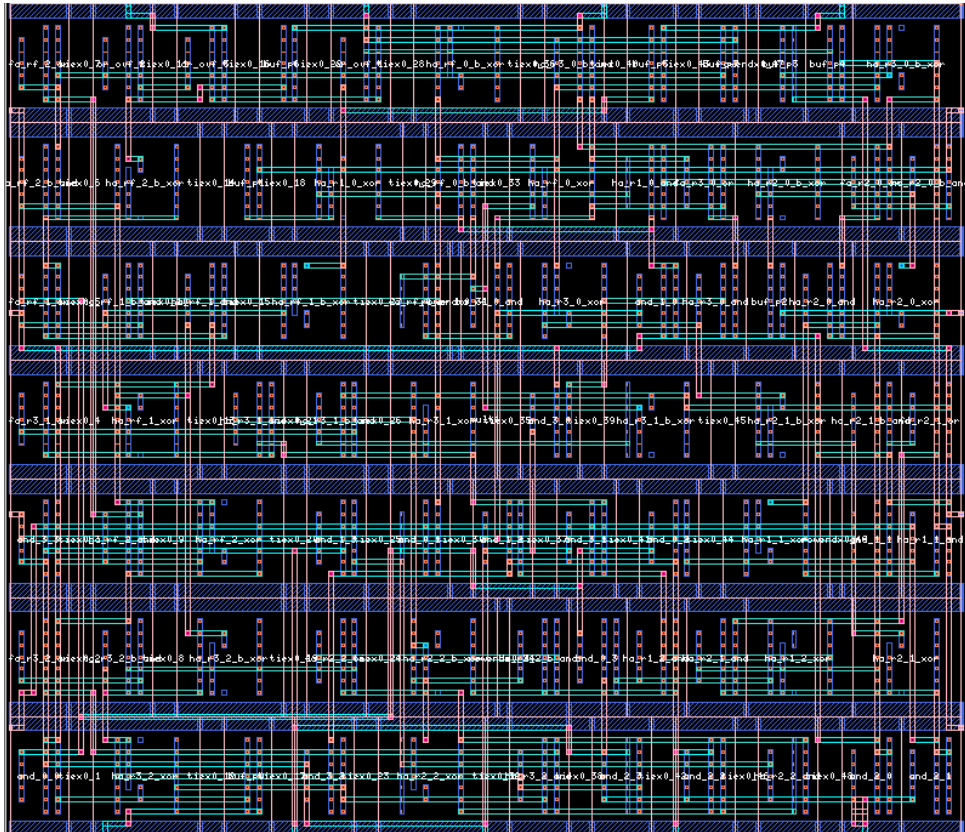


Figura 1 — Layout físico do mult4 após posicionamento (ocp) e roteamento (nero), capturado do graal: 127 células em 7 fileiras.

6. Layout físico e atraso de propagação

O netlist foi posicionado (ocp) e roteado (nero) em um layout físico real (Figura 1). O atraso de propagação foi medido, não estimado: `find_delay.sh` roda o conjunto exaustivo de 256 casos no asimut com tempos de acomodação decrescentes entre forçar as entradas e checar as saídas, até a simulação começar a errar.

Acomodação	Erros
13 ns	0
12 ns	1
11 ns	8
10 ns	25

O resultado é 13 ns, usando o atraso declarado nos modelos comportamentais da sxlib (after 1000 ps em cada célula), para a transição de pior caso ($0 \times 0 \rightarrow 15 \times 15$). Como conferência, o netlist extraído do layout roteado (cougar -t -ac, 578 transistores, capacitâncias parasitas reais) foi simulado no spiceopus com um modelo de transistor genérico, não calibrado a nenhuma fábrica real (a tecnologia do Alliance usada aqui só tem regras geométricas, sem parâmetros elétricos). Sob esse modelo a mesma transição se estabiliza em cerca de 1,8 ns — número apenas ilustrativo, já que depende do modelo emprestado, mas na mesma ordem de grandeza.

7. Conclusão

O multiplicador foi implementado, verificado exhaustivamente (256/256), posicionado, roteado e teve seu atraso de propagação medido. A maior parte do esforço não foi o projeto lógico, que é direto, mas o ambiente: binários quebrados, caminhos de arquivo não documentados e um simulador SPICE sem modo não-interativo, todos contornados nos scripts do repositório.

Repositório: github.com/Lucasnobrega38/mult4-alliance-cad